

SPEAKER BRIEFING · SMART INDIA HACKATHON

What's actually running inside POLAR-EMS

A working reference for presenting the live demo: what the simulation is really computing, what every number on screen means, how the two control algorithms differ, and how this maps onto a genuine polar-station engineering problem.

SIMULATION	CONTROLLERS	PRIORITY TIERS	DEMO SCENARIOS	LIVE PAGES
100% client-side	2 (baseline + AI)	4	8	7

01 What the website is actually doing

It is not a slideshow of pretty numbers. It is a small, running physics-and-rules model of a diesel/wind/solar/battery microgrid, ticking forward in simulated time, with two independent control algorithms competing to serve the same load from the same weather.

Every second of real time, the page advances the simulated clock by a few hours (configurable 1×/4×/12×) and re-runs the full pipeline: generate this tick's weather → compute renewable output → compute station load → run the rule-based baseline controller → run the AI-heuristic optimizer → apply the hard safety layer → check for anomalies → update forecasts → accumulate the green-audit totals → redraw whichever tab is open. Nothing is pre-recorded; every chart you point at during the demo is being generated live in the browser as you talk.

WHY THIS MATTERS FOR JUDGES

If someone asks "is this just a mockup with fake numbers," the honest and correct answer is: no — it's a deterministic simulation with real physical relationships (power curves, seasonal solar cycles, battery energy balance, fuel-burn curves), and every KPI is derived from that running state, not hardcoded.

The one sentence to open with

"Diesel is scarce and expensive, the internet can vanish for weeks, and losing power at -45°C is a safety event — so POLAR-EMS forecasts load and renewables, optimizes the diesel/battery/wind/solar mix in real time, and keeps making safe decisions with zero cloud connectivity."

02 The five layers, and where the demo lives

The whole point of the architecture is that each layer can keep working if the one above it disappears. The live page demonstrates layers 1–4 running entirely in-browser; layer 5 (cloud) is simulated by the Offline Autonomy tab.

LAYER	WHAT IT DOES HERE	SURVIVES
1• Field sensors	Weather, generation, load and battery/diesel telemetry — produced by the physical model each tick	Total isolation
2• Edge gateway	Ingests that telemetry and runs every model below, entirely client-side	Zero internet
3• AI engine	Load/renewable forecasting, dispatch optimization, anomaly detection	Sparse data
4• Dispatch & safety	Executes the chosen controller's decision; hard safety rules can override it	AI/model failure
5• Cloud portal	Represented by the sync outbox on the Offline Autonomy tab	Days-to-weeks offline

03 Every parameter, explained

Grouped exactly as they appear across the seven tabs. Use this as the glossary if a judge points at a number and asks "what's that."

Weather & season model

PARAMETER	MEANING	HOW IT'S GENERATED
Temperature	Outside air temp, $^{\circ}\text{C}$	Seasonal sine curve (-45°C deep winter to -5°C deep summer) + daily swing + noise

PARAMETER	MEANING	HOW IT'S GENERATED
Wind speed	m/s at hub height	Smoothed random walk (autocorrelated) around a seasonal mean, so it drifts realistically instead of jittering tick to tick
Irradiance	Solar input, W/m²	Seasonal sun potential × time-of-day factor × cloud cover, forced to zero through the modeled polar-night window
Season label	Polar night / winter / summer / midnight sun	Derived from the same seasonal function that drives solar potential

Renewable generation

PARAMETER	MEANING	HOW IT'S GENERATED
Wind output	kW from the turbine array	Standard turbine power curve: 0 below 3.5 m/s cut-in, cubic ramp to rated output at 12 m/s, 0 above 25 m/s cut-out
Solar output	kW from the PV array	$\text{Irradiance} \div 1000 \times \text{array capacity} \times 92\% \text{ system efficiency}$
Renewable fraction	% of load served without diesel	Renewables actually absorbed by load, divided by total load, per tick and cumulatively

Station load — four priority tiers

This hierarchy is enforced everywhere shedding happens: amenities go first, then general power, then research, and life-support/heating are only ever touched by the hard safety override, never by an ordinary dispatch decision.

TIER	LOAD	TYPICAL SHARE	SHED ORDER
P1	Life support + heating (heating rises as temperature falls)	~40–45%	Never, except critical-SOC override
P2	Research / lab equipment	~20–25%	Third
P3	General station power	~15–20%	Second
P4	Amenities	~8–10%	First

Battery

PARAMETER	MEANING
SOC	State of charge, % of 500 kWh usable capacity
Charge / discharge rate	Capped at 150 kW in either direction
Reserve floor	Baseline controller: fixed 20%. AI controller: 20–32%, raised when the 3-hour renewable forecast is weak
Critical floor	10% — below this the hard safety layer takes over regardless of which controller is active
Temperature / cycles / health	Tracked for the anomaly & maintenance page; excess cycling and thermal drift degrade the health score

Diesel generator

PARAMETER	MEANING
Rated output	300 kW
Specific fuel consumption (SFC)	Litres of diesel per kWh delivered — base 0.30 L/kWh, with a penalty modeled when the generator runs below ~50% load (real diesel gensets are least efficient at low load, which is exactly why the AI controller avoids starting it for small shortfalls)
Fuel tank	12,000 L capacity
Runtime hours	Cumulative hours running — the number the optimizer is explicitly trying to minimize

Forecast metrics (Forecast tab)

PARAMETER	MEANING
Horizon	How far ahead the prediction targets: 1h, 6h, 12h or 24h
MAE	Mean absolute error between predicted and realized load, in kW — "on average, how far off"
RMSE	Root-mean-square error — same units, penalizes large misses more than MAE
Samples scored	How many predictions have actually been resolved against reality so far in this run

Optimizer / dispatch outputs (Optimizer tab)

PARAMETER	MEANING
Renewables served	kW of the current load coming directly from wind + solar
Battery	kW flowing into (+) or out of (–) the battery this tick
Diesel	kW currently being delivered by the generator
Load shed	kW of non-critical load intentionally dropped (should be 0 in normal operation)
Why (reason)	A plain-English sentence the controller generates for its own decision every tick — this is the single most important feature for the "is this explainable" question

Anomalies & health (Anomalies tab)

PARAMETER	MEANING
Severity	Critical / warning / info
Detected vs. expected	The actual reading next to what the physical model says it should have been
Subsystem health (0–100%)	Battery, turbine array, generator — decays when anomalies fire, slowly recovers otherwise

Offline & sync (Offline Autonomy tab)

PARAMETER	MEANING
Outbox count	Telemetry records generated while offline, waiting to reach the cloud
Local subsystem status	Telemetry / AI forecasting / dispatch optimizer / safety controller — all shown ACTIVE independent of connectivity, which is the entire point of the tab

Green Audit (cumulative KPIs)

PARAMETER	MEANING
Diesel consumed / saved	Litres burned by the AI-run station vs. the shadow rule-based baseline, since the run started
CO ₂ avoided	Fuel saved × 2.68 kg CO ₂ per litre (standard diesel emission factor)
Renewable fraction	Share of all energy served by wind + solar over the whole run
Runtime reduction	Generator hours saved vs. baseline

PARAMETER	MEANING
What-if projection	A capacity-ratio scaling of the currently measured fuel rate — explicitly labeled a projection, not a re-simulation

04 How the internals actually work

This is the part that answers "so what's the AI actually doing" without hand-waving.

Two controllers, same inputs, every tick

Both controllers see the identical weather draw and identical load for that tick, so any difference in outcome is attributable to the control logic itself — not to lucky weather. This is what makes the fuel-savings number on the Optimizer and Green Audit tabs a measured result rather than an assumption.

● Rule-based baseline

Logic

- Serve load from renewables first, always
- Any surplus charges the battery; any deficit discharges it
- Fixed 20% reserve floor, no matter the weather outlook
- If the battery can't cover the rest, start diesel for exactly the shortfall
- No forecast awareness — purely reactive to the current tick

● POLAR-EMS optimizer

Logic

- Same renewables-first, battery-second waterfall
- Reserve floor moves to 32% when the next-3-hour renewable forecast is weak, 20% when it's strong — it holds back charge when it expects to need it
- When diesel must start, it deliberately runs a little above the immediate shortfall to also recharge the battery, so it doesn't have to cold-start again soon
- Diesel is avoided specifically because it's least fuel-efficient at low partial load — the SFC curve models this directly

THE JUDGE-FACING PUNCHLINE

The baseline is not a strawman — it's a fully functional, sensible controller. The optimizer beats it only because it looks ahead and times diesel starts smarter. That's the actual value proposition of

"AI-driven": not that rules are broken, but that a forecast-aware policy burns less fuel and runs the generator fewer hours for the identical job.

Hard safety layer — runs after both controllers, every tick

1 Critical SOC override

If battery SOC drops below 10% regardless of which controller is driving, diesel is forced to full rated output and non-critical tiers are shed — this check cannot be disabled by either controller.

2 AI/sensor failure override

Triggering the "AI service failure" scenario doesn't just change a label — it actually re-routes the live dispatch through the rule-based controller's logic for as long as the fault is active. The mode badge (

● AI OPTIMIZED

 /

● SAFE FALLBACK

) always reflects which logic is actually driving the station, not a cosmetic state.

Anomaly detection — statistical, not a black box

Every anomaly check compares an actual reading against what the physical model says it should be, and only fires when the gap crosses a threshold. That's deliberate: a judge can ask "how do you know it's anomalous" and the answer is always a specific expected-vs-detected comparison, never "the model said so."

CHECK	TRIGGER
Turbine underperformance	Actual wind output >25% below the power-curve prediction for the current wind speed
Battery thermal / degradation	Temperature drifting outside −10°C to 15°C with an accelerated rise
Generator efficiency drift	Specific fuel consumption >15% above the baseline value
Sensor fault	An implausible reading (e.g. a dropout to zero) inconsistent with surrounding telemetry

Forecasting — a statistical baseline, plus two genuinely trained models

The default forecast for a future hour blends the seasonal expectation for that hour (same time of year, same time of day) with the current deviation from that expectation, decayed the further out the prediction reaches — a fixed formula with no learned parameters:

```
predicted(t+h) = seasonal_baseline(t+h) + current_deviation × e-h/14 + noise
```

Alongside it, two real trained models run in parallel and are scored the same way. In the browser: an online linear regression (Widrow-Hoff/LMS) that takes one gradient step every simulated tick against that session's own telemetry — its live weights are shown on the Forecast tab, starting at zero and visibly moving. On the backend: a scikit-learn `GradientBoostingRegressor`, trained once at server startup on a train/test split of synthetic data generated from the same load model, with a real held-out MAE (not a training-set number) exposed at `/api/forecast/ml` and in the CLI. Accuracy for all three is verified, not assumed — every prediction is time-stamped, and once the simulation reaches that real moment, MAE/RMSE are recalculated against what actually happened.

Fuel & CO₂ accounting

```
fuel_used (L) = diesel_kW × SFC(load_fraction) × hours
fuel_saved = fuel_used_baseline - fuel_used_AI
CO2_avoided (kg) = fuel_saved × 2.68
```

2.68 kg CO₂ per litre is a standard published diesel combustion emission factor, not a tuned number — state that plainly if asked.

05 How this maps onto the real problem

Every feature in the demo exists because of a specific, named constraint of running a polar station — not because it looked good on a dashboard.

Fuel is scarce & costly

Every litre is shipped in at extreme cost. **Answer:** forecast-aware dispatch that avoids starting diesel for shortfalls the battery can cover, and avoids running it inefficiently at low partial load.

Internet is unreliable

Satellite links can drop for weeks. **Answer:** every control layer runs at the edge; the Offline Autonomy tab proves telemetry, forecasting, optimization and safety never pause, only the cloud sync queues.

Failure is a safety event

Losing heat at -45°C is life-threatening. **Answer:** a strict, independently-enforced load-priority hierarchy plus a hard-coded override that cannot be overruled by either controller.

Equipment degrades in extreme cold

Batteries, turbines and sensors drift under stress.

Answer: statistical anomaly detection flags degradation early, with a plain-English recommendation, before it becomes a failure.

"Why should we trust an AI here?"

Answer: it's not opaque — every dispatch decision stores the reason it was made, and the safety layer sits outside the AI entirely, so trust doesn't depend on the model being right 100% of the time.

Judges expect rigor, not vibes

Answer: the baseline vs. optimized comparison runs on identical conditions, so the fuel/CO₂ savings shown are a measured delta, not an assumed one — and change if the scenario changes.

06 Judge Q&A cheat sheet

Straight answers, no overclaiming — this is the tone that reads as engineering maturity rather than a pitch. Grouped so you can find the right answer fast if you're cold-called on it.

Frontend & backend development

What's the frontend actually built with?

Plain HTML, CSS and JavaScript — no framework. It's one self-contained file that runs the entire simulation client-side (weather model, dispatch logic, charts, everything), chosen deliberately so it's portable with zero build step and zero dependencies: it runs the same way as a static file, on Vercel, or inside a sandboxed preview.

What's the backend actually built with?

Python, using FastAPI for the REST API and a plain Python class for the simulation engine. It's deployed on Render, exposes interactive Swagger docs at </docs>, and ships with a terminal client ([cli.py](#)) that talks to it over plain HTTP — nothing exotic.

Why two separate simulations instead of one shared codebase?

The website has to keep working with zero backend — that's the entire point of the "edge autonomy" story for a station with unreliable internet. The Python backend is a second, independently-runnable implementation of the same physical model, built to prove a real server, a CLI, and API-driven control also work, the way an actual deployment would need.

Is there a database?

Not yet, by design for this prototype — the backend keeps recent history in memory rather than persisting to disk. Adding SQLite or Postgres for long-term storage is explicit future work, not something we're pretending already exists.

How are the frontend and backend connected?

A REST API over HTTPS. An optional "Live Backend" panel on the site polls the FastAPI server every 1.5 seconds. It's deliberately decoupled: the main dashboard runs its own simulation and needs no backend at all, and this panel is an additive proof that the same logic also runs as a real server you can hit from a terminal.

What was the hardest technical problem you actually hit?

Browsers block an HTTPS page from calling a plain HTTP address ("mixed content"). We hit this directly — backend on localhost, frontend on HTTPS via Vercel — diagnosed it correctly, and fixed it by giving the backend real HTTPS on Render rather than trying to bypass browser security.

About the "AI" — the honest version

Where exactly is the AI?

In three concrete, pointable-to places. (1) A scikit-learn `GradientBoostingRegressor` (150 trees), trained offline with a genuine held-out train/test split — 5.4 kW mean error on data it never saw during training — then **exported and deployed to run real inference client-side, in the browser, on the live site**: its trees were serialized to a JSON file and a small hand-written JS function re-walks them, verified to reproduce scikit-learn's own predictions bit-for-bit before it ever shipped. Zero server call at prediction time. (2) The identical trained model also runs server-side in the FastAPI backend, at `/api/forecast/ml`. (3) An online linear regression that trains itself live, one gradient step per simulated tick, directly from this session's own telemetry — you can watch its weights update on the Forecast tab. All three are visible side by side on that tab, scored against realized actuals. What's deliberately *not* AI: the dispatch optimizer and the hard safety override, which are auditable, deterministic rules — the next answer explains why that's a design choice, not a gap.

What AI/ML is actually running here?

A mix, and we're specific about which parts are genuinely trained: a forecast-aware heuristic optimizer for dispatch (a constrained decision policy, not a trained model) and threshold-based anomaly detection (physically-expected vs. actual) are explainable rules, not ML. Load forecasting has three real implementations, all visible on the Forecast tab: the frontend runs an online linear regression (Widrow-

Hoff/LMS) that trains itself one step per tick directly from session telemetry; a scikit-learn `GradientBoostingRegressor` trained offline on a held-out split runs server-side in the backend at `/api/forecast/ml`; and that same trained model is also exported and deployed to run genuine inference entirely client-side in the browser, with no backend required. No deep learning anywhere — that's a deliberate choice, not a gap, argued in the next answer.

Isn't that a weak claim for an "AI-Driven" project?

It's a defensible claim, not a weak one: for a safety-relevant system, a decision policy you can audit and verify is more trustworthy than a black box. "AI-driven" here means reasoning under uncertainty with a forecast to beat a fixed rule — and we prove that delta directly with the baseline-vs-optimized comparison, rather than asserting model sophistication we don't have.

Did you use AI coding tools to build this?

Yes, and there's no reason to dodge that — we used an AI coding assistant to speed up implementation: boilerplate, porting logic between Python and JavaScript, debugging. The architecture, the algorithms, the safety rules, and every design decision are ours; the assistant is a tool, the same way an IDE is a tool. What matters is that we can explain every line of the dispatch logic, and we can.

Why not deep learning or reinforcement learning for dispatch?

A constrained, rule-explainable heuristic is verifiable and safe to reason about for a system where a wrong decision has real consequences. That matters more here than raw model sophistication.

What happens if the AI is just wrong, or unavailable?

The hard safety layer sits outside the optimizer and can't be overridden by it. Trigger the "AI service failure" scenario live — the dashboard visibly switches to SAFE FALLBACK and dispatch keeps running on deterministic rules.

Can you show the AI actually reasoning, live?

Yes — open the Optimizer tab or run the CLI and look at the "Why" field: every dispatch tick stores a plain-English reason, e.g. "battery discharged 75 kW because SOC is above the 32% reserve floor set by a weak 3-hour renewable forecast." It's not a black box; you can read its reasoning as it happens.

How the website and programs are actually running

Walk me through what happens the moment the site loads.

A tick loop starts immediately in the browser: generate this tick's weather → compute renewables → compute load → run both dispatch controllers → apply the safety layer → check for anomalies → update forecasts → redraw. No server call is required for any of that — it's a self-contained physics-and-rules simulation.

Is the offline-mode demo faked?

No — toggling it flips a real boolean the simulation checks every tick. Telemetry keeps recording into a real local outbox array while offline, and reconnecting drains that queue with an actual animated multi-step sync, mirroring how store-and-forward works on a real intermittent link.

What happens if I refresh the page?

The simulation resets to a fresh run — state lives only in browser memory, not in local storage. That's a known, stated limitation, not something to be caught off guard by if asked.

What happens with zero internet for weeks?

Nothing changes operationally — telemetry, forecasting, optimization and safety all run at the edge. Only the cloud sync queues records in a local outbox until connectivity returns, then uploads in a batch.

How is it actually hosted?

Frontend: static HTML on Vercel's CDN. Backend: a FastAPI service on Render, containerized and health-checked over HTTPS. Source lives on GitHub. Nothing hand-managed beyond that — both platforms redeploy from a git push.

The real-world problem & benefit

Is this real station data?

No, and the site says so explicitly. It's a synthetic physical model (power curves, seasonal solar/polar-night cycles, priority-weighted load, battery/diesel dynamics) — not random numbers, and not a live feed. That's a deliberate, stated engineering choice for a hackathon build.

What real problem does this solve, concretely?

A polar station burns expensive, shipped-in diesel inefficiently because nobody is continuously re-optimizing the wind/solar/battery/diesel mix against a weather forecast, and losing power there is a

safety event, not an inconvenience. This system automates that continuous re-optimization while guaranteeing a hard safety floor underneath it.

Who would actually use this?

Station operations staff and engineers responsible for keeping life-support, heating and research equipment running — the dashboard is built for someone monitoring the microgrid day to day, not for a data scientist.

How is fuel/CO₂ savings calculated?

Two controllers run in parallel on identical weather and load. Fuel saved is the litre difference between them; CO₂ avoided is that difference times the standard 2.68 kg/L diesel emission factor.

What's the single most valuable feature for a station operator?

Arguably the safety fallback, not the optimizer: it's the guarantee that a model failure, a sensor fault, or a lost connection never means losing power to life-support or heating — the optimizer's fuel savings are the upside, the fallback is the floor.

What would it take to run this on a real station?

Swap the synthetic telemetry generator for real sensor/BMS/genset feeds into the same edge pipeline — the forecasting, optimization, safety and anomaly-detection logic are all designed to consume exactly that shape of data already.

What's the limitation you're most upfront about?

The backend has no persistent database yet — recent history lives in memory, not on disk, so a server restart loses it. And every trained model here is trained on synthetic data generated from our own physical model, not a real station's historical sensor logs, because no such public dataset exists for this use case; the honest framing is "the methodology transfers, the numbers are illustrative." Both are named as future work rather than glossed over — a judge respects a clearly-scoped prototype far more than an overstated one.

POLAR-EMS · internal briefing document · for Smart India Hackathon presentation use